



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS

**Reporte proyecto 1 (Implementación de protocolo
de chat utilizando sockets TCP).**

PRESENTA

Bruno Bernardo Soto Lugo
426008646

PROFESOR

Canek Peláez Valdés

AYUDANTES

Miriam Torres Bucio
Cynthia Lizbeth Sánchez Urbano

Índice general

1. Introducción	3
2. Análisis	4
2.1. Requisitos	4
3. Diseño	6
3.1. Diagramas UML	6
3.1.1. Diagrama de secuencia (Proceso de autenticación)	6
4. Desarrollo	7

Capítulo 1

Introducción

El presente documento detalla el proceso de implementación de un protocolo que especifica una forma de comunicación chat. El sistema está compuesto por dos componentes principales: un cliente con interfaz gráfica desarrollado en **Kotlin** con el framework **Compose Multiplatform** y un servidor desarrollado en lenguaje **C**.

El objetivo principal del proyecto es implementar un protocolo de comunicación basado en objetos JSON que permita la identificación de usuarios, cambio de estados (Activo, Ausente, Ocupado), envío de mensajes privados y públicos, así como la gestión de salas de chat dinámicas. El desarrollo siguió el modelo de cascada.

Las etapas fueron:

1. **Análisis:** Definición de requisitos funcionales basados en las acciones del usuario.
2. **Diseño:** Arquitectura del sistema, diseño de la base de datos en memoria y diagramas UML.
3. **Desarrollo:** Codificación del servidor en C y la interfaz en Kotlin.
4. **Pruebas:** Validación de cada comando del protocolo contra el manual de especificaciones.

Capítulo 2

Análisis

En esta fase se definieron las capacidades que la implementación del sistema debe ofrecer según el protocolo.

2.1. Requisitos

A partir del análisis del protocolo, se identificaron los siguientes componentes:

- **Gestión de Identidad (IDENTIFY):** El sistema debe permitir al usuario unirse con un nombre único (con máximo 8 caracteres). El servidor debe validar la existencia previa y notificar a otros usuarios la entrada de nuevos miembros.
- **Gestión de Estado (STATUS):** Los usuarios pueden alternar entre **ACTIVE**, **AWAY** y **BUSY**. Cada cambio debe disparar un evento **NEW_STATUS** para los demás clientes conectados.
- **Comunicación (TEXT, PUBLIC_TEXT y ROOM_TEXT):**
 - Mensajes privados que solo el destinatario recibe.
 - Mensajes públicos dirigidos a todos los usuarios en la conexión global.
 - Mensajes dirigidos a todos los usuarios en una sala.
- **Gestión de Salas (ROOMS):** El sistema debe permitir crear salas (con máximo 16 caracteres), invitar usuarios y gestionar pertenencia a sala. Se requiere validación de que el usuario esté invitado antes de unirse.
- **Obtención de usuarios (USERS y ROOM_USERS):** El usuario puede obtener la lista de usuarios que están conectados globalmente o en una sala a la que pertenece.
- **Desconexión (DISCONNECT):** Manejo de limpieza de recursos cuando un cliente pierde conexión o decide salir, notificando su salida a las salas que pertenece y globalmente.

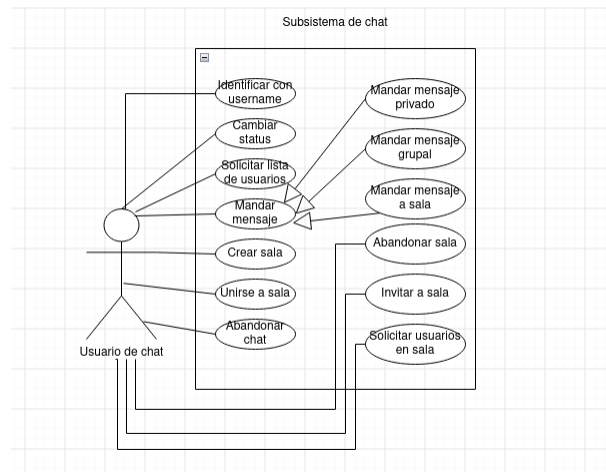


Figura 2.1: Diagrama del análisis (Use Case).

Capítulo 3

Diseño

El diseño implementa una arquitectura dividida en cliente y servidor con el objetivo de desacoplar las partes del programa y permitir que diferentes clientes puedan conectarse a un mismo servidor. El servidor actúa como un gestor de múltiples conexiones simultáneas. El cliente se encarga de la representación visual y la administración de la lógica local.

3.1. Diagramas UML

A continuación se detallarán en diagramas UML las estructuras usadas para implementar el protocolo.

3.1.1. Diagrama de secuencia (Proceso de autenticación)

El siguiente diagrama representa la secuencia que sigue el cliente y el servidor cuando un usuario busca autenticarse.

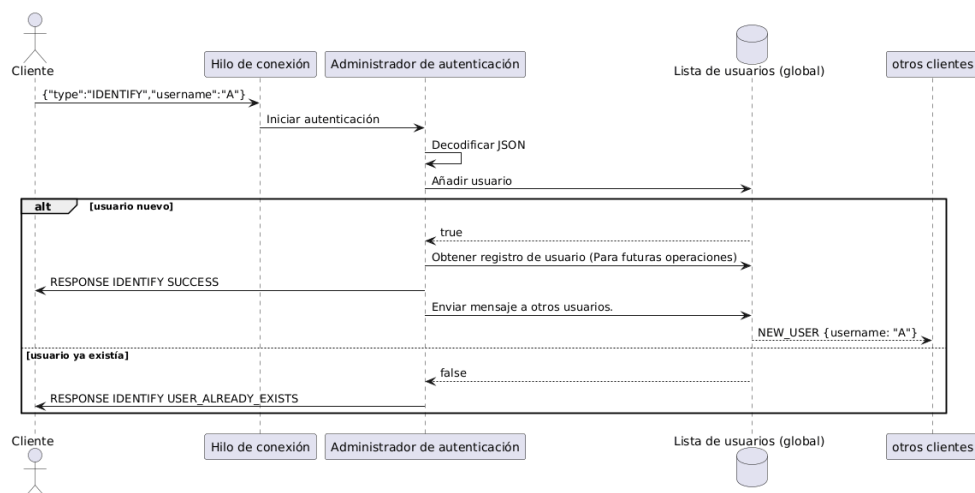


Figura 3.1: Diagrama de la autenticación.

Capítulo 4

Desarrollo